



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

**SEGG-U1-P1 — OWASP Top 10: Investigación
y análisis de vulnerabilidades**

Contenido

1. Tabla comparativa OWASP	3
2. Casos Reales	5
Caso 1: El desastre de TalkTalk (2015).....	5
Caso 2: LulzSec y el hackeo a Sony Pictures (2011)	6
3. Código Vulnerable vs Seguro (JavaScript / Node.js).....	6
Ejemplo 1: A03 - Injection (SQLi)	6
Ejemplo 2: A01 - Broken Access Control (IDOR).....	6
Ejemplo 3: A07 - Identification and Authentication Failures	7
4. Ranking Personal de Criticidad	8
1. A03 - Inyección (Injection)	8
2. A01 - Broken Access Control (Control de Acceso Roto).....	8
3. A06 - Componentes Vulnerables y Desactualizados	8
4. A02 - Fallas Criptográficas	8
5. A07 - Fallas de Identificación y Autenticación.....	9
5. Conclusión	9

1. Tabla comparativa OWASP

Nombre	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
A01: Broken Access Control	Acciones realizadas fuera de los permisos asignados.	Cambiar la URL para ver datos de otro usuario (IDOR).	Exposición, modificación o destrucción de datos.	Controles estrictos en backend; principio de mínimo privilegio.
A02: Cryptographic Failures	Falla al proteger datos sensibles en tránsito o en reposo.	Interceptar tráfico HTTP no cifrado para robar claves.	Fraude, robo de identidad o exposición de datos.	Cifrar datos, forzar HTTPS/TLS, evitar algoritmos obsoletos.
A03: Injection	Datos no confiables ejecutados como parte de un comando.	Ingresa ' OR '1'='1 en login para evadir autenticación.	Robo total de la base de datos o ejecución remota.	Usar consultas parametrizadas, validar y sanitizar inputs.
A04: Insecure Design	Fallas en el diseño y la arquitectura desde el inicio.	Recuperación de contraseña sin límite de intentos.	Depende del fallo, suele generar compromisos masivos.	Modelado de amenazas, limitar tasa de peticiones (Rate Limiting).

Nombre	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
A05: Security Misconfiguration	Configuraciones inseguras, incompletas o por defecto.	Dejar credenciales por defecto activas (admin/admin).	Acceso no autorizado al sistema o red interna.	Proceso de <i>hardening</i> , deshabilitar funciones innecesarias.
A06: Vulnerable Components	Uso de librerías de terceros con fallos conocidos.	Utilizar una versión antigua y vulnerable de Log4j.	Toma de control total del servidor host (RCE).	Monitorear y actualizar dependencias constantemente (Snyk).
A07: Auth Failures	Problemas en la gestión de sesiones y contraseñas.	Ataques de fuerza bruta o relleno de credenciales.	Secuestro de cuentas de usuarios legítimos.	Autenticación Multifactor (MFA), políticas de contraseñas fuertes.
A08: Software Integrity Failures	Falta de protección contra modificaciones no autorizadas.	Descargar actualización de un servidor comprometido.	Distribución masiva de malware, accesos no autorizados.	Verificar firmas digitales, usar repositorios confiables.
A09: Logging & Monitoring Failures	Ausencia de registros para detectar ataques activos.	Atacante prueba contraseñas días sin	El atacante mantiene acceso prolongado	Registrar todos los eventos de acceso/errores, usar SIEM.

Nombre	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
		generar alertas.	sin ser detectado.	
A10: Server-Side Request Forgery	El servidor lee URLs externas sin validar el destino.	Forzar al servidor a leer metadatos internos de AWS.	Acceso a redes internas o credenciales en la nube.	Validar URLs de usuarios; usar listas de permitidos (Allow-lists).

2. Casos Reales

Caso 1: El desastre de TalkTalk (2015)

Contexto y cómo ocurrió la explotación En octubre de 2015, TalkTalk (uno de los gigantes de las telecomunicaciones en Reino Unido) sufrió una brecha masiva que los dejó muy mal parados en la industria. Lo más increíble del caso es que no usaron tecnología de punta para hackearlos. Los atacantes aprovecharon un error de inyección SQL extremadamente básico en unas páginas web antiguas que la empresa había heredado tras comprar otra compañía, y que simplemente se les olvidó parchear o auditar.

El impacto real Los atacantes metieron comandos SQL a través de la web y se descargaron la base de datos. Lo que más llamó la atención fue que el principal responsable era un adolescente de 17 años.

- Robaron datos sensibles de casi 157,000 clientes (nombres, direcciones, correos y datos bancarios parciales).
- La oficina de protección de datos del Reino Unido (ICO) los multó con £400,000 libras por negligencia en su seguridad.
- Entre la fuga de clientes, multas y reparación de la red, el incidente le costó a TalkTalk cerca de £42 millones de libras.

Caso 2: LulzSec y el hackeo a Sony Pictures (2011)

Contexto y cómo ocurrió la explotación En el verano de 2011, el infame colectivo de hackers LulzSec decidió ir por Sony Pictures. La empresa tenía una página web promocional con nula validación de datos en sus formularios. LulzSec aplicó una inyección SQL clásica para engañar a la base de datos y obtener acceso directo a los servidores internos.

El impacto real Lo que descubrieron los atacantes fue un regalo: Sony no estaba aplicando ningún tipo de cifrado o *hashing* a sus bases de datos. LulzSec extrajo más de un millón de registros que incluían contraseñas, correos, fechas de nacimiento y direcciones postales, todo almacenado en texto plano.

- El grupo publicó la información en internet y usó sus redes sociales para burlarse de Sony, exhibiendo que su seguridad era de nivel "amateur".
- Sony enfrentó demandas colectivas millonarias, tuvo que dar de baja múltiples servidores y la confianza pública en su manejo de datos quedó severamente dañada.

3. Código Vulnerable vs Seguro (JavaScript / Node.js)

Ejemplo 1: A03 - Injection (SQLi)

Código Vulnerable:

El input del usuario se concatena directamente, permitiendo inyección

```
1  const user_id = req.body.id;
2  const query = "SELECT * FROM users WHERE id = " + user_id;
3  db.execute(query);
4  |
```

Código Seguro (Consultas parametrizadas):

Se usan placeholders (?) para separar los datos de la instrucción SQL

```
1  const user_id = req.body.id;
2  const query = "SELECT * FROM users WHERE id = ?";
3  db.execute(query, [user_id]);
```

Ejemplo 2: A01 - Broken Access Control (IDOR)

Código Vulnerable:

Solo verifica que el usuario esté logueado, pero no si es DUEÑO del documento:

```

1  app.get('/documentos/:docId', (req, res) => {
2    if (req.isAuthenticated()) {
3      const doc = db.getDocument(req.params.docId);
4      res.send(doc);
5    }
6  });
7

```

Código Seguro:

Verifica que el usuario logueado tenga permisos sobre ese documento específico

```

1  app.get('/documentos/:docId', (req, res) => {
2    if (req.isAuthenticated()) {
3      const doc = db.getDocument(req.params.docId);
4      if (doc.owner_id === req.user.id) {
5        res.send(doc);
6      } else {
7        res.status(403).send('Acceso denegado');
8      }
9    }
10  });

```

Ejemplo 3: A07 - Identification and Authentication Failures

Código Vulnerable:

Guardar contraseñas en texto plano

```

1  function registerUser(username, password) {
2    const query = "INSERT INTO users (username, password) VALUES (?, ?)";
3    db.execute(query, [username, password]);
4  }

```

Código Seguro:

Usar un algoritmo de hashing fuerte como bcrypt

```

1  const bcrypt = require('bcrypt');
2
3  async function registerUser(username, password) {
4    const saltRounds = 12;
5    const hashedPassword = await bcrypt.hash(password, saltRounds);
6    const query = "INSERT INTO users (username, password) VALUES (?, ?)";
7    db.execute(query, [username, hashedPassword]);
8  }

```

4. Ranking Personal de Criticidad

1. A03 - Inyección (Injection)

Considero que las inyecciones son el riesgo número uno por la facilidad con la que se pueden automatizar los ataques y el nivel catastrófico de su impacto. Un simple error de validación en un formulario de login puede darle a un atacante acceso íntegro a la base de datos. Esto no solo permite la exfiltración masiva de información confidencial, sino también la modificación o destrucción total de los datos, lo que puede paralizar por completo las operaciones de una empresa en cuestión de segundos.

2. A01 - Broken Access Control (Control de Acceso Roto)

Lo coloco en segundo lugar porque es un error humano extremadamente frecuente en el diseño de la lógica de negocio. A diferencia de otras vulnerabilidades, los escáneres de seguridad automatizados rara vez logran detectar estas fallas porque no entienden el contexto de la aplicación. Si el backend no verifica estrictamente los permisos en cada petición, un usuario común puede acceder a paneles de administración o ver información privada de otros clientes con solo alterar un parámetro en la URL (como en los ataques IDOR).

3. A06 - Componentes Vulnerables y Desactualizados

Hoy en día, casi ninguna aplicación se construye desde cero; el desarrollo moderno depende enormemente de librerías y dependencias de código abierto. Posiciono esta vulnerabilidad en el top 3 por el "efecto dominó" que genera. Como demostró el incidente de *Log4j*, un solo fallo descubierto en una librería popular puede comprometer de la noche a la mañana a millones de aplicaciones y servidores a nivel mundial, incluso si el código interno de nuestra aplicación estaba perfectamente escrito.

4. A02 - Fallas Criptográficas

La falta de un cifrado adecuado es crítica porque cuando fallan las defensas perimetrales, la criptografía es la última barrera para proteger los datos. El impacto aquí va mucho más allá de lo técnico; deriva en multas legales severas y daños irreversibles a la reputación. Si un atacante logra acceder a una base de datos (por ejemplo, a través de una inyección SQL), el hecho de que las contraseñas, datos bancarios o información de salud estén en texto plano o protegidos con algoritmos obsoletos convierte un incidente grave en un desastre total.

5. A07 - Fallas de Identificación y Autenticación

Aunque construyamos sistemas robustos, el usuario final suele ser el eslabón más débil, reciclando contraseñas en múltiples plataformas. Lo incluyo en el ranking porque si nosotros, como desarrolladores, no implementamos protecciones obligatorias como la Autenticación Multifactor (MFA) o bloqueos por intentos fallidos, le facilitamos el trabajo a los ciberdelincuentes. Mediante ataques automatizados de relleno de credenciales (*credential stuffing*), los atacantes pueden secuestrar miles de cuentas legítimas sin necesidad de encontrar una brecha compleja en nuestro código.

5. Conclusión

Investigar el OWASP Top 10 para esta práctica me dejó una lección muy clara: la gran mayoría de los hackeos millonarios que vemos en las noticias no son obra de genios utilizando tecnología imposible, sino el resultado directo de errores básicos que nosotros, como programadores, cometemos por ignorancia, descuido o prisa. Cuando uno empieza en el desarrollo de software, es fácil pensar que programar bien es simplemente lograr que el código funcione y cumpla con los requerimientos visuales o de negocio. Sin embargo, entender esta lista te cambia la perspectiva por completo. Te das cuenta de que dejar un simple campo de texto sin validar o no revisar los permisos en una ruta de la API puede tirar abajo todo el trabajo de un equipo y, peor aún, arruinarle la vida a las personas al exponer sus datos privados. El OWASP Top 10 no es solo un documento técnico; es un catálogo de nuestras peores prácticas y un recordatorio constante de la responsabilidad que tenemos en nuestras manos.

En mi futura vida profesional, este conocimiento cambia radicalmente la forma en la que voy a encarar mis proyectos. Ya no veo la ciberseguridad como una tarea aburrida que se deja para el final, esperando a que un auditor encuentre los fallos. Ahora entiendo que debo programar a la defensiva desde la primera línea de código. A nivel práctico, esto significa que voy a automatizar el uso de consultas parametrizadas para evitar inyecciones, dudaré siempre de la información que venga del lado del cliente y seré estricto con los controles de acceso. Conocer el OWASP Top 10 me da una ventaja real en el mercado laboral: me enseña a pensar un poco como un atacante para poder defender mi trabajo, permitiéndome entregar no solo software que se vea bien, sino sistemas en los que la gente realmente pueda confiar.